A1 Code structure

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track A: Databricks pipeline engineering 2
A1 Code structure 3
A1-L1 Single-purpose functions and clear names 3
A1-L2 Type hints and docstrings a reviewer can trust 7
A1-L3 Config over hardcoded values, and PEP 8 with ruff 12
Track A best-practice checklist 18
Appendix: solutions and answer keys 19

Track A: Databricks pipeline engineering

Build Databricks pipelines that are correct, fast, observable and tested.

A1 Code structure

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Functions, type hints, docstrings, naming, config files and PEP 8.

A1-L1 Single-purpose functions and clear names

Concept

A function should do **one** of five jobs: read, transform, validate, write or log. A sixth kind, the **orchestrator**, calls the others in order and does nothing else.

Single-purpose functions pay off in three ways:

- **Tests.** A transform that takes a DataFrame and returns a DataFrame runs on a local SparkSession with three rows. A function that reads a table, transforms it and writes it needs a live catalog.
- **Reuse.** `filter_in_scope()` serves gold, QA and ad hoc analysis. A 200-line `process()` serves one notebook.
- **Change.** When the supplier renames a column, you edit one transform. Nothing else moves.

A useful test: describe the function in one sentence without "and". "Aggregate reach by wave and subgroup" passes. "Read silver and compute reach and write gold" fails three times.

Names carry the contract. Follow the standard from your reference project:

Object	Convention	Example
Function	`verb_noun`, snake_case	`check_table_health`
Class	PascalCase noun	`CheckResult`
Constant	UPPER_SNAKE_CASE	`MAX_WEIGHT_CV`
Bronze table	`[tracker]_raw`	`nat_raw`
Silver table	`[tracker]_respondents`	`nat_respondents`
Gold table	`[tracker]_metrics`	`nat_metrics`
Job	`[tracker]_[layer]_pipeline`	`nat_silver_pipeline`

A name must match what the function does. `get_` and `check_` promise no side effects. A `get_unprocessed_waves()` that also deletes rows breaks that promise, and the next reader calls it without knowing.

Why it matters

A gold notebook holds one 350-line cell: read silver, filter, weight, aggregate, write, log. A change request asks for a new subgroup. The developer edits the aggregation, and a variable named `df` later in the cell now points at the new frame. The write step saves the unfiltered data. Gold publishes reach figures that include out-of-scope respondents for two waves.

Nobody could test the cell without writing to production tables, so nobody tested it. With `filter_in_scope()` and `aggregate_reach()` as separate functions, a three-row unit test catches the mistake before the merge.

Worked example: weak code

```
1  def process(t, w):
2      df = spark.table(t).filter(F.col("wave_id") == w)
3      df = df.filter(F.col("in_scope") == True)
4      df = df.groupBy("wave_id",
       "region").agg(F.sum("final_weight").alias("weighted_base"))
5      df.write.mode("overwrite").saveAsTable("research_gold.nat_metrics")
6      print("done", df.count())
```

1. **Line 1.** The name says nothing, and `t` and `w` hide what the arguments are.
2. **Line 3.** `== True` trips ruff E712. Filter on the boolean column directly.
3. **Line 5.** The target table is hardcoded inside a function that also reads and transforms.
4. **Line 6.** Logging by `print`, and a second full scan to count rows.

Worked example: strong code

```
1  def read_wave(spark: SparkSession, table: str, wave_id: str) -> DataFrame:
2      """Read one wave from a silver table."""
3      return spark.table(table).filter(F.col("wave_id") == wave_id)
4
5
2  def filter_in_scope(df: DataFrame) -> DataFrame:
7      """Keep respondents flagged as in scope."""
3      return df.filter(F.col("in_scope"))
9
10
11 def aggregate_base(df: DataFrame) -> DataFrame:
12     """Weighted base at wave x region grain."""
13     return df.groupBy("wave_id",
       "region").agg(F.sum("final_weight").alias("weighted_base"))
14
15
4  def run_gold(spark: SparkSession, silver: str, gold: str, wave_id: str) -
   > None:
17     """Orchestrate: read, filter, aggregate, write."""
```

```
18      metrics = aggregate_base(filter_in_scope(read_wave(spark, silver,
        wave_id)))
5       metrics.write.mode("overwrite").option("replaceWhere", f"wave_id =
        '{wave_id}'").saveAsTable(gold)
```

1. **Line 1.** Read only. The session and the table name come in as parameters.
2. **Line 6.** A pure transform: DataFrame in, DataFrame out. Testable with three rows.
3. **Line 8.** Boolean column used directly.
4. **Line 16.** The orchestrator calls the steps in order and holds no logic of its own.
5. **Line 19.** Overwrite only this wave, so a rerun never wipes other waves.

Each strong function passes the one-sentence test. The weak one needs "and" four times.

Recap card

- **The five jobs a pipeline function does, one at a time** Read, transform, validate, write, log. An orchestrator calls them in order.
- **The one-sentence test** Describe the function without using "and". If you need "and", split it.
- **Function, class and constant naming** check_table_health, CheckResult, MAX_WEIGHT_CV.
- **What does a get_ or check_ prefix promise?** No side effects. The function reads and returns. It never writes or deletes.

Practice

Exercise 1. Multiple choice (level 1)

You write a function that returns a list of `CheckResult` records for one table. Which name follows the standard?

- a) `CheckTableHealth`
- b) `check_table_health`
- c) `checkTableHealth`
- d) `CHECK_TABLE_HEALTH`

Why?

- r1) Functions use snake_case and start with a verb.
- r2) Shorter names run faster.

Exercise 2. Fill the gap (level 2)

Fill in the object names for tracker `nat`, following the naming standard.

```
BRONZE_TABLE = "research_bronze._____(1)"
SILVER_TABLE = "research_silver._____(2)"
GOLD_TABLE = "research_gold._____(3)"
JOB_NAME = "_____(4)"
```

Exercise 3. Spot the bug (level 3)

This function has a name that promises no side effects. One line breaks that promise. Tap it, then fix the function so it only returns the waves.

What you need to know.

The code chains five DataFrame and session methods (PySpark 4.0):

- `df.select("wave_id")` keeps only the named columns.
- `.distinct()` drops duplicate rows.
- `a.subtract(b)` returns the distinct rows of `a` that are not in `b`.
- `.collect()` brings the rows back to Python as a list of `Row` objects. Read a field with `row.wave_id`.
- `spark.sql("...")` runs one SQL statement, such as a `SELECT` or a `DELETE`.

```
a = spark.createDataFrame([("W1",), ("W2",), ("W2",)], "wave_id STRING")
b = spark.createDataFrame([("W1",)], "wave_id STRING")
[row.wave_id for row in a.subtract(b).collect()]   # ['W2']
```

```
def get_unprocessed_waves(spark: SparkSession, bronze: str, silver: str,
    tracker_id: str) -> list[str]:
    """Return wave ids in bronze but not yet in silver for one tracker."""
    bronze_waves = spark.table(bronze).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
    silver_waves = spark.table(silver).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
    waves = [row.wave_id for row in
    bronze_waves.subtract(silver_waves).collect()]
    spark.sql(f"DELETE FROM {silver} WHERE wave_id IN ({','.join(repr(w)
    for w in waves)})")
    return waves
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Refactor (level 4)

Split the transform logic out of `process`. Write two pure functions:

- `filter_in_scope(df)` keeps rows where `in_scope` is true.
- `aggregate_base(df)` returns `wave_id`, `region` and `weighted_base` (sum of `final_weight`).

Neither function may read or write a table. Add type hints and docstrings.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Keep respondents flagged as in scope.
def filter_in_scope(df: DataFrame) -> DataFrame: ...

# Weighted base at wave x region grain.
def aggregate_base(df: DataFrame) -> DataFrame: ...
```

Starter code:

```python
def process(t, w):
    df = spark.table(t).filter(F.col("wave_id") == w)
    df = df.filter(F.col("in_scope") == True)
    df = df.groupBy("wave_id",
   "region").agg(F.sum("final_weight").alias("weighted_base"))
    df.write.mode("overwrite").saveAsTable("research_gold.nat_metrics")
```

Exercise 5. Architecture design (level 5)

You split a gold notebook into functions. Where does each piece of work belong?

Options: Config file, Read function, Transform function, Orchestrator.

1. The name of the gold table for tracker nat
2. `spark.table(silver).filter(wave_id == ...)`
3. Summing weights by wave and region
4. Calling read, filter, aggregate and write in turn, then logging the run

Sources

- ruff rule E712 flags comparisons to True or False. (verified: Reference README lists it; ruff 0.16.9 flags `x == True` under E712 in this repository.)
- replaceWhere overwrites only matching rows in a Delta table. (verified: Run locally on delta-spark 4.0.1 (VERSIONS.md, Track A facts).)

A1-L2 Type hints and docstrings a reviewer can trust

Concept

A type hint states a contract. `keys: list[str]` tells the caller, the reviewer and mypy that `keys` is a list of column names. A bare `list` says "some list". No hint says nothing.

Use the modern forms, valid from Python 3.10:

- Built-in generics: `list[str]`, `dict[str, int]`, `tuple[str, ...]`.
- Optional values: `str | None = None`. Never `str = None`, which claims the value is always a string.

- Fixed sets of strings: `Literal["PASS", "WARN", "FAIL"]` from `typing`. mypy then rejects `"pass"` or `"FAILED"`.
- A return type on every function, including `-> None`.

Run `mypy --strict` in CI. Strict mode refuses functions without hints, so gaps cannot creep back.

A docstring explains what hints cannot: meaning, units, grain and failure modes. Use the Google style:

```
def weighted_share(df: DataFrame, flag_col: str) -> float | None:
    """Weighted share of rows where flag_col is 1.

    Args:
        df: Respondents with final_weight.
        flag_col: 0/1 column to share.

    Returns:
        Share between 0 and 1, or None when the total weight is 0.

    Raises:
        ValueError: If flag_col is missing from df.
    """
```

The first line fits on one line and says what the function returns or does. Args, Returns and Raises follow. A reviewer reads the docstring, then checks the body against it. A mismatch between the two is a bug in one of them.

PySpark is not fully typed, and mypy cannot see column names. Hints still catch the most common mistake: passing the wrong kind of object.

Why it matters

The reference `upsert_silver` takes `merge_keys: list`. A caller passes the string `"respondent_id"` instead of `["respondent_id"]`. Python iterates a string character by character, so the join builds `t.r = s.r AND t.e = s.e AND ...`. The MERGE fails with a column-not-found error at 02:00, and the on-call engineer spends an hour reading the wrong part of the stack trace.

With `merge_keys: list[str]`, mypy flags the call in CI: a `str` is not a `list[str]`. The bug never reaches the scheduler.

Worked example: weak code

```
def log_pipeline_run(pipeline_name, layer, rows_processed, status,
    error_message: str = None):
    """Append one record per pipeline run to the governance run log."""
    record = {
        "pipeline_name": pipeline_name,
        "layer": layer,
```

```
6          "rows_processed": rows_processed,
4          "status": status,
8          "error_message": error_message,
9      }
10     return record
```

1. **Line 1.** Claims a string, defaults to None. Reference defect 7.
2. **Line 1.** No hints on four parameters and no return type. mypy --strict rejects the function.
3. **Line 2.** The docstring says it appends, but the function returns a dict. One of them is wrong.
4. **Line 7.** `status` accepts any string, so `"sucess"` slips into the run log.

Worked example: strong code

```
1  from typing import Literal
2
1  RunStatus = Literal["SUCCESS", "FAILED"]
4  Layer = Literal["bronze", "silver", "gold"]
5
6
2  def build_run_record(
8      pipeline_name: str,
9      layer: Layer,
10     rows_processed: int,
11     status: RunStatus,
3      error_message: str | None = None,
4  ) -> dict[str, str | int | None]:
14     """Build one run log record.
15
16     Args:
17         pipeline_name: Job name, for example nat_silver_pipeline.
18         layer: Medallion layer the run wrote to.
19         rows_processed: Rows written by the run.
20         status: Final status of the run.
21         error_message: Error text for a failed run, otherwise None.
22
23     Returns:
24         A record ready to append to the run log table.
25     """
26     return {
27         "pipeline_name": pipeline_name,
28         "layer": layer,
29         "rows_processed": rows_processed,
30         "status": status,
31         "error_message": error_message,
32     }
```

1. **Line 3.** A named Literal type. mypy rejects any other string at every call site.
2. **Line 7.** The name now matches the job: it builds a record. Writing it is a separate function.

3. **Line 12.** Optional, stated honestly.
4. **Line 13.** The return type matches what the body returns.

The strong version tells mypy, the reviewer and the caller the same story. The weak version tells three different ones.

Recap card

- **Optional parameter that defaults to None** param: str | None = None. Never param: str = None.
- **A fixed set of string values** Literal["PASS", "WARN", "FAIL"] from typing.
- **Google docstring sections** One-line summary, then Args, Returns, Raises.
- **Why merge_keys: list[str] and not list?** mypy then rejects a string, which Python would split into characters.

Practice

Exercise 1. Multiple choice (level 1)

A function takes an optional error message that defaults to `None`. Which signature is correct for Python 3.11 with `mypy --strict`?

- a) `def log_run(error_message: str = None) -> None:`
- b) `def log_run(error_message: str | None = None) -> None:`
- c) `def log_run(error_message=None) -> None:`
- d) `def log_run(error_message: None = str) -> None:`

Why?

- r1) The hint must include None, because None is a value the parameter takes.
- r2) Type hints change how Python runs the function.

Exercise 2. Predict the output (level 2)

The reference code builds a MERGE condition like this. A caller passes a string instead of a list. What does this print?

```
1 merge_keys = "id"
2 condition = " AND ".join(f"t.{k} = s.{k}" for k in merge_keys)
3 print(condition)
```

Exercise 3. Fill the gap (level 2)

Complete the signature. `status` accepts only `"PASS"`, `"WARN"` or `"FAIL"`, and the function returns nothing.

```python
from typing import _____(1)

def record_check(name: str, status: _____(2)["PASS", "WARN", "FAIL"]) ->
    _____(3):
    ...
```

Exercise 4. Write the function (level 3)

Write `build_merge_condition(keys, target="t", source="s")`. It returns a condition such as `t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id`.

- Raise `TypeError` when `keys` is a string, not a list.
- Raise `ValueError` when `keys` is empty.
- Add type hints and a Google-style docstring with Args, Returns and Raises.

Your code must pass these checks:

- Join one `t.k = s.k` clause per key with `AND`.
- Use the `target` and `source` aliases you receive.
- A string must raise TypeError. Otherwise it is split into characters.
- An empty key list must raise ValueError. An empty condition matches nothing and inserts every row.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Build a MERGE ON condition that matches every key column.
def build_merge_condition(
    keys: list[str],
    target: str = 't',
    source: str = 's',
) -> str: ...
```

Starter code:

```python
def build_merge_condition(keys, target="t", source="s"):
    ...
```

Exercise 5. Refactor (level 4)

Type this function. Return one of `"SKIP"`, `"WARN"` or `"PASS"` as a `Literal`, accept a missing `change`, and add a Google-style docstring. Keep the behaviour.

Your code must pass these checks:

- The behaviour changed. None gives SKIP, above tolerance gives WARN, otherwise PASS.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Classify a wave-over-wave change in a share.
def classify_change(
    change: float | None,
    tolerance: float = 0.1,
) -> ChangeStatus: ...
```

Starter code:

```python
def classify_change(change, tolerance=0.10):
    if change is None:
        return "SKIP"
    return "WARN" if change > tolerance else "PASS"
```

Sources

- Iterating a string in a join builds one clause per character. (verified: Run in Python 3.11 in this repository.)
- The X | Y union syntax works in annotations from Python 3.10. (verified: Python 3.10 release notes (PEP 604); used throughout this repository on Python 3.11.)

A1-L3 Config over hardcoded values, and PEP 8 with ruff

Concept

Pipeline code should contain logic. Everything that changes between environments, trackers or contract versions belongs in configuration:

- catalog, schema and table names
- column maps and type maps
- thresholds, such as `weight_sum_min` or a stability tolerance
- contract rules and their severity

Keep configuration in one of two places. Versioned JSON or YAML files in the repository suit values that change with a code release. Registry tables in Unity Catalog suit values that analysts maintain, such as the Variable Registry column map.

Load configuration once, at the start of the run, into a typed object. A frozen dataclass or a pydantic model works. Validate on load: every required key present, `weight_sum_min` below `weight_sum_max`, names non-empty. Fail at 00:01 with "missing key: silver_table", not at 02:47 with a KeyError deep in a transform.

Then pass the object, or the values from it, into your functions. A function that reads a global config dictionary is as hard to test as one with hardcoded strings.

Style is part of the same discipline. PEP 8 is the Python style guide. **ruff** checks it and formats code in one fast tool:

- `ruff check .` finds errors and style problems, such as undefined names (F821) or `== True` (E712).
- `ruff format .` rewrites layout consistently. `ruff format --check .` fails CI when a file needs formatting.

Settle the rules once in `pyproject.toml` under `[tool.ruff]`, run both commands in CI, and stop debating layout in reviews.

Why it matters

A developer copies the production silver notebook to test a change in the dev workspace. The notebook hardcodes `research_prod.research_silver.nat_respondents`. The dev run MERGEs half-finished test data into the production table. The morning report shows impossible reach figures, and the team restores the table from Delta history.

With the catalog in a config file per environment, the dev run writes to `research_dev`. The same code runs in both places, and only the config changes.

Worked example: weak code

```
1 def check_weight_sum(spark, wave_id):
2     total = (
1         spark.table("research_prod.research_silver.nat_respondents")
4         .filter(F.col("wave_id") == wave_id)
5         .agg(F.sum("final_weight")).collect()[0][0]
6     )
2     if total < 45000000 or total > 55000000:
3         raise ValueError("weight sum out of range")
```

1. **Line 3.** The production table is typed into the function. A dev run writes, or here reads, production.
2. **Line 7.** Contract thresholds are buried in code. A new contract version needs a code change in every notebook that repeats them.
3. **Line 8.** The error hides the numbers. You cannot tell how far out of range the total is.

Worked example: strong code

```
1 import json
```

```
from dataclasses import dataclass
from pathlib import Path


@dataclass(frozen=True)
class WeightContract:
    """Weight total limits from the data contract."""

    silver_table: str
    weight_sum_min: float
    weight_sum_max: float

    def __post_init__(self) -> None:
        if self.weight_sum_min >= self.weight_sum_max:
            raise ValueError("weight_sum_min must be below weight_sum_max")


def load_contract(path: Path) -> WeightContract:
    """Load and validate the contract file. Fails fast on a missing key."""
    raw = json.loads(path.read_text())
    return WeightContract(
        silver_table=raw["silver_table"],
        weight_sum_min=float(raw["weight_sum_min"]),
        weight_sum_max=float(raw["weight_sum_max"]),
    )


def weight_sum_in_range(total: float, contract: WeightContract) -> bool:
    """True when the weight total is inside the contract limits."""
    return contract.weight_sum_min <= total <= contract.weight_sum_max
```

1. **Line 6.** Frozen: no code can change a limit halfway through a run.
2. **Line 14.** Validation on load. An impossible contract fails before any data moves.
3. **Line 23.** A missing key raises KeyError here, at load, with the key's name.
4. **Line 29.** Pure logic with the limits passed in. Test it with plain numbers, no Spark needed.

The strong version runs unchanged in dev and prod. The contract file decides the table and the limits.

Recap card

- **What belongs in configuration?** Anything that changes between environments, trackers or contract versions: names, maps, thresholds, rules.
- **When do you validate configuration?** Once, on load, before any data moves. Fail fast with the key's name.

- **Why not raw.get("catalog", "research_prod")?** A default hides a missing key and can point a run at production.
- **The two ruff commands for CI** ruff check . and ruff format --check .

Practice

Exercise 1. Multiple choice (level 1)

Which of these belongs in configuration rather than in pipeline code?

- a) The formula for weighted reach: weighted reach divided by weighted base
- b) The catalog name, `research_prod` or `research_dev`
- c) The order of steps: read, validate, transform, write
- d) The decision to MERGE rather than append in silver

Why?

- r1) It changes between environments while the logic stays the same.
- r2) Short strings belong in configuration.

Exercise 2. Predict the output (level 2)

What does this print?

What you need to know.

`@dataclass(frozen=True)` makes every instance read-only. Assigning to a field after the object exists raises `dataclasses.FrozenInstanceError`, and the field keeps its old value.

`try` runs a block. When the block raises an exception named in `except`, Python runs the `except` block instead of stopping, then carries on after it.

```python
from dataclasses import FrozenInstanceError, dataclass

@dataclass(frozen=True)
class Limit:
    value: int

limit = Limit(5)
try:
    limit.value = 9
except FrozenInstanceError:
    print("read-only")
```

```python
from dataclasses import FrozenInstanceError, dataclass


@dataclass(frozen=True)
class RunConfig:
```

```
    catalog: str


cfg = RunConfig("research_dev")
try:
    cfg.catalog = "research_prod"
except FrozenInstanceError:
    print("blocked")
print(cfg.catalog)
```

Exercise 3. Fill the gap (level 2)

Complete the two CI commands. The first finds lint errors. The second fails when a file needs reformatting, without changing it.

```
ruff _____(1) .
ruff _____(2) --check .
```

Exercise 4. Write the function (level 4)

Write `load_run_config(raw)`. It takes a dict parsed from JSON and returns a frozen `RunConfig` dataclass with `catalog` (str), `tracker` (str) and `stability_tolerance` (float).

- Raise `KeyError` naming the key when one is missing.
- Raise `ValueError` when `stability_tolerance` is not between 0 and 1.
- Add a property `silver_table` that returns `f"{catalog}.research_silver.{tracker}_respondents"`.

Your code must pass these checks:

- A valid dict must load into RunConfig with the same values.
- `silver_table` must build the three-level name from catalog and tracker.
- A missing key must raise KeyError that names the key.
- A tolerance outside 0 to 1 must raise ValueError. 10 means 1000 percentage points.
- The config must be frozen, so no code can change it mid-run.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Settings for one pipeline run.
class RunConfig: ...

# Build a validated RunConfig from parsed JSON.
def load_run_config(raw: dict[str, object]) -> RunConfig: ...
```

Starter code:

```
from dataclasses import dataclass

```

```
3
4  @dataclass(frozen=True)
5  class RunConfig:
6      ...
7
8
9  def load_run_config(raw: dict[str, object]) -> RunConfig:
10     ...
```

Exercise 5. Spot the bug (level 5)

The team moved every table name into `RunConfig`. A dev run still reads production. Tap the line, then fix the function.

```
1  def weighted_base_by_wave(spark: SparkSession, cfg: RunConfig) ->
     DataFrame:
2      """Weighted base per wave from the configured silver table."""
3      silver = spark.table(cfg.silver_table)
4      scope = spark.table("research_prod.research_governance.scope_rules")
5      in_scope = silver.join(scope, "region").filter(F.col("in_scope"))
6      return
     in_scope.groupBy("wave_id").agg(F.sum("final_weight").alias("weighted_base"))
```

Write down the line number of the bug. Then write the fix.

Sources

- ruff check reports F821 (undefined name) and E712 (comparison to True); ruff format --check fails when a file needs formatting. (verified: ruff 0.16.9 run in this repository.)
- Assigning to a frozen dataclass field raises FrozenInstanceError. (verified: Python 3.11 standard library, run in this repository.)

Track A best-practice checklist

☐ Every function has type hints, a docstring and one job.

☐ No SparkSession is created at import time. Functions receive spark as a parameter.

☐ Functions are imported as F: `from pyspark.sql import functions as F`.

☐ Table names, column maps and thresholds come from config or registry tables.

☐ Every table states its grain in a comment or table property.

☐ Bronze appends. Silver deduplicates the source, then MERGEs. Gold aggregates by every grain column.

☐ Every run writes one record to the run log table with an explicit schema.

☐ CRITICAL checks raise. WARN checks log and continue.

☐ Every division has a guard for zero and null.

☐ Counts come from one agg pass, not repeated count() calls.

☐ Incremental loads compare source and target on the same keys.

☐ Row filters use a SQL UDF with SET ROW FILTER. Grants go to groups.

☐ Tests run on a local SparkSession in CI before every merge.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

A1-L1 Single-purpose functions and clear names

A1-L1 exercise 1

Hints. (1) Look at the naming table in the concept. (2) Classes and constants have their own styles. (3) Functions: lower case, words joined by underscores, verb first.

Answer. b) `check_table_health` Because: r1) Functions use snake_case and start with a verb.

Functions use `verb_noun` in snake_case. PascalCase is for classes and UPPER_SNAKE_CASE for constants, so the style alone tells a reader what kind of object they are looking at.

Common wrong answers.

- `a`: PascalCase marks a class, such as `CheckResult`. A function uses snake_case.
- `c`: camelCase is the Java and JavaScript convention. PEP 8 and your standard use snake_case for functions.
- `d`: UPPER_SNAKE_CASE marks a constant, such as `MAX_WEIGHT_CV`.
- `b|r2`: Right name, wrong reason. Name length has no effect on speed. The convention signals what kind of object it is.

A1-L1 exercise 2

Hints. (1) Tables follow `[tracker]_[content]`. (2) Bronze holds raw deliveries, silver holds respondents, gold holds metrics. (3) The job pattern is `[tracker]_[layer]_pipeline`.

Answer.

```
BRONZE_TABLE = "research_bronze.nat_raw"
SILVER_TABLE = "research_silver.nat_respondents"
GOLD_TABLE = "research_gold.nat_metrics"
JOB_NAME = "nat_silver_pipeline"
```

`nat_raw`, `nat_respondents` and `nat_metrics` name what each table holds. The schema carries the layer. Jobs use `nat_silver_pipeline`.

Common wrong answers.

- g1 = `nat_bronze`: The layer is already in the schema name, `research_bronze`. The table name says what it holds: `nat_raw`.
- g4 = `silver_nat_pipeline`: Jobs start with the tracker: `[tracker]_[layer]_pipeline`. Notebooks start with the layer.
- g2 = `nat_silver`: Silver tables are named after their grain: one row per respondent, so `nat_respondents`.

A1-L1 exercise 3

Hints. (1) What does the word `get` promise the caller? (2) Which line changes a table? (3) Delete line 6. Deleting data belongs in its own, clearly named function.

Answer. The bug is on line 6.

Fixed version:

```
1 def get_unprocessed_waves(spark: SparkSession, bronze: str, silver: str,
    tracker_id: str) -> list[str]:
2     """Return wave ids in bronze but not yet in silver for one tracker."""
3     bronze_waves = spark.table(bronze).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
4     silver_waves = spark.table(silver).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
5     waves = [row.wave_id for row in
    bronze_waves.subtract(silver_waves).collect()]
6     return waves
```

Line 6 deletes rows from silver inside a function named `get_`. A caller who only wants the list of waves loses data. Remove the line. If you need that clean-up, give it its own function, such as `delete_waves()`, and call it from the orchestrator.

Common wrong answers.

- Line 5: `collect()` on a short list of wave ids is fine. Look for a line that changes data.
- Line 4: Filtering silver by tracker is correct. It fixes a known defect in the reference code.

A1-L1 exercise 4

Hints. (1) Each function takes `df: DataFrame` and returns a DataFrame. (2) Move line 3 into `filter_in_scope` and line 4 into `aggregate_base`. (3) Drop `== True`, and leave `spark.table` and `.write` out of both functions.

Answer.

```python
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def filter_in_scope(df: DataFrame) -> DataFrame:
    """Keep respondents flagged as in scope."""
    return df.filter(F.col("in_scope"))


def aggregate_base(df: DataFrame) -> DataFrame:
    """Weighted base at wave x region grain."""
    return df.groupBy("wave_id",
      "region").agg(F.sum("final_weight").alias("weighted_base"))
```

Pure transforms take a DataFrame and return one. Reading and writing stay in the orchestrator, so the transforms run in a unit test with a handful of rows.

A1-L1 exercise 5

Hints. (1) Values that change between environments are config. (2) Anything that touches the catalog is a read or a write. (3) The orchestrator only sequences the other functions.

Answer.

- Table names are configuration. Load them once and pass them in.
- `spark.table(...)` is a read. It takes the session and the table name as parameters.
- Summing weights is a pure transform.
- The orchestrator calls each step and logs the run. It holds no business logic.

Config holds names and thresholds. Reads touch the catalog. Transforms turn one DataFrame into another. The orchestrator calls them in order and logs the outcome.

Common wrong answers.

- table_name = `orchestrator`: A table name typed into the orchestrator is still hardcoded. Put it in config, so dev and prod runs point at different tables.
- weights = `orchestrator`: If the orchestrator holds the aggregation, you cannot test it without running the whole pipeline. Make it a transform.
- spark_table = `transform`: A transform receives a DataFrame. Reading a table needs a session and a catalog, which is the read function's job.

A1-L2 Type hints and docstrings a reviewer can trust

A1-L2 exercise 1

Hints. (1) The default is None. Does the hint allow None? (2) Python 3.10 added the `X | Y` union syntax. (3) `str | None = None`.

Answer. b) `def log_run(error_message: str | None = None) -> None:` Because: r1) The hint must include None, because None is a value the parameter takes.

`str | None = None` says the parameter is a string or None, and defaults to None. Python ignores hints at run time. mypy and readers rely on them.

Common wrong answers.

- `a`: `str = None` claims the value is always a string. mypy strict mode rejects the implicit Optional.
- `c`: No hint at all. mypy --strict rejects unannotated parameters.
- `d`: This makes the type `None` and the default the class `str`. It is backwards.
- `b|r2`: Right signature, wrong reason. Python ignores hints at run time. They exist for tools and readers.

A1-L2 exercise 2

Hints. (1) What does `for k in "id"` loop over? (2) A string is a sequence of characters. (3) Two characters, two conditions, joined by AND.

Answer.

```
1 t.i = s.i AND t.d = s.d
```

A string is iterable, so the loop yields `i` and `d`. The condition names two columns that do not exist. With `merge_keys: list[str]`, mypy flags the call before it runs.

Common wrong answers.

- `t.id = s.id`: That is what the caller intended. Iterating a string yields its characters, `i` then `d`.
- `TypeError`: Python raises no error. A string is iterable, which is why the bug is silent until Spark runs the MERGE.

A1-L2 exercise 3

Hints. (1) The type for a fixed set of values lives in `typing`. (2) Its name means the values are written out literally. (3) `Literal`, and `None` for the return type.

Answer.

```
from typing import Literal

def record_check(name: str, status: Literal["PASS", "WARN", "FAIL"]) -> None:
    ...
```

`Literal["PASS", "WARN", "FAIL"]` restricts the value to those three strings. `-> None` states the function returns nothing.

Common wrong answers.

- g1b = `Enum`: An Enum is a class you define. Inline string choices use `Literal[...]` from `typing`.
- g2 = `void`: Python has no `void`. A function that returns nothing is annotated `-> None`.
- g1b = `str`: `str` accepts any string, so `"pass"` or `"FAILED"` would slip through. `Literal` lists the allowed values.

A1-L2 exercise 4

Hints. (1) Check the type first with `isinstance(keys, str)`, then check for an empty list. (2) Build each clause with an f-string that uses `target` and `source`. (3) `" AND ".join(f"{target}.{k} = {source}.{k}" for k in keys)`.

Answer.

```
def build_merge_condition(keys: list[str], target: str = "t", source: str = "s") -> str:
    """Build a MERGE ON condition that matches every key column.

    Args:
        keys: Columns that identify one target row.
        target: Alias of the target table.
        source: Alias of the source DataFrame.

    Returns:
        The condition, for example "t.id = s.id AND t.wave = s.wave".

    Raises:
        TypeError: If keys is a string instead of a list.
        ValueError: If keys is empty.
    """
    if isinstance(keys, str):
```

```
        raise TypeError("keys must be a list of column names, not a string")
    if not keys:
        raise ValueError("keys must name at least one column")
    return " AND ".join(f"{target}.{k} = {source}.{k}" for k in keys)
```

The hint `list[str]` lets mypy catch a string at the call site. The run-time `TypeError` catches callers that skip mypy, such as a notebook. The `ValueError` stops an empty condition, which would match nothing and insert every row.

A1-L2 exercise 5

Hints. (1) Import `Literal` from `typing` and name the return type. (2) `change` is `float | None`. `tolerance` is `float`. (3) Keep `if change is None:`. Truthiness would turn a change of 0.0 into SKIP.

Answer.

```
from typing import Literal

ChangeStatus = Literal["SKIP", "WARN", "PASS"]


def classify_change(change: float | None, tolerance: float = 0.10) -> ChangeStatus:
    """Classify a wave-over-wave change in a share.

    Args:
        change: Absolute change in the share, or None when a share is missing.
        tolerance: Largest change that still passes.

    Returns:
        SKIP when change is None, WARN above tolerance, otherwise PASS.
    """
    if change is None:
        return "SKIP"
    return "WARN" if change > tolerance else "PASS"
```

The Literal return type means mypy rejects a typo such as `\"PASSED\"` at every caller. `float | None` records that a change can be missing. The `is None` test keeps a real change of 0.0 as PASS.

A1-L3 Config over hardcoded values, and PEP 8 with ruff

A1-L3 exercise 1

Hints. (1) What changes when the same code runs in dev and in prod? (2) Logic stays the same. Names and limits change. (3) The catalog differs between environments.

Answer. b) The catalog name, `research_prod` or `research_dev` Because: r1) It changes between environments while the logic stays the same.

The catalog name varies by environment. The formula, the step order and the write strategy are logic, which belongs in reviewed, tested code.

Common wrong answers.

- `a`: The formula is business logic. It is the same in every environment, so it belongs in a tested function.
- `c`: The step order is the pipeline's design. It lives in the orchestrator and changes only with a code review.
- `d`: MERGE versus append is a design decision about idempotence, not a setting.
- `b|r2`: Right choice, wrong reason. Length is irrelevant. Configuration holds what varies between environments, trackers or contract versions.

A1-L3 exercise 2

Hints. (1) What does `frozen=True` do to attribute assignment? (2) The exception fires before the value changes. (3) Two lines are printed.

Answer.

```
1 blocked
2 research_dev
```

`frozen=True` makes assignment raise `FrozenInstanceError`, so `blocked` prints and the catalog stays `research_dev`. No code can switch a run to production halfway through.

Common wrong answers.

- `research_prod`: A frozen dataclass rejects assignment after creation, so the value stays `research_dev`.
- `blocked research_prod`: The assignment raised before it changed anything. The value stays `research_dev`.

A1-L3 exercise 3

Hints. (1) One command lints, one formats. (2) The lint command's name means inspecting code. (3) `ruff check .` and `ruff format --check .`.

Answer.

```
ruff check .
ruff format --check .
```

`ruff check .` reports lint errors such as F821 and E712. `ruff format --check .` exits non-zero when any file would be reformatted, which fails CI without touching the code.

Common wrong answers.

- g1 = `lint`: ruff has no `lint` command. Linting runs with `ruff check`.
- g2 = `fmt`: The formatter command is `ruff format`. Add `--check` so CI reports instead of rewriting.
- g1 = `format`; g2 = `check`: Swapped. `ruff check` lints, `ruff format --check` verifies formatting.

A1-L3 exercise 4

Hints. (1) Declare three fields on a frozen dataclass, then add a `@property` for the table name. (2) Index the dict with `raw["key"]`. `.get` with a default hides a missing key. (3) Check `0 <= tolerance <= 1` before you build the object.

Answer.

```
from dataclasses import dataclass


@dataclass(frozen=True)
class RunConfig:
    """Settings for one pipeline run."""

    catalog: str
    tracker: str
    stability_tolerance: float

    @property
    def silver_table(self) -> str:
        """Three-level name of the silver table."""
        return f"{self.catalog}.research_silver.{self.tracker}
    _respondents"


def load_run_config(raw: dict[str, object]) -> RunConfig:
    """Build a validated RunConfig from parsed JSON.

```

```
    Args:
        raw: Parsed configuration.

    Returns:
        The frozen configuration.

    Raises:
        KeyError: If a required key is missing.
        ValueError: If stability_tolerance is outside 0 to 1.
    """
    tolerance = float(str(raw["stability_tolerance"]))
    if not 0 <= tolerance <= 1:
        raise ValueError(f"stability_tolerance
{tolerance} is outside 0 to 1")
    return RunConfig(
        catalog=str(raw["catalog"]),
        tracker=str(raw["tracker"]),
        stability_tolerance=tolerance,
    )
```

Indexing with `raw[\"key\"]` fails at load with the key's name. Defaults such as `raw.get(\"catalog\", \"research_prod\")` are worse than no config: a missing value silently points the run at production.

A1-L3 exercise 5

Hints. (1) Search the function for a catalog name. (2) Every table name must come from `cfg`. (3) Add `scope_table` to RunConfig and read `cfg.scope_table`.

Answer. The bug is on line 4.

Fixed version:

```
def weighted_base_by_wave(spark: SparkSession, cfg: RunConfig) ->
DataFrame:
    """Weighted base per wave from the configured silver table."""
    silver = spark.table(cfg.silver_table)
    scope = spark.table(cfg.scope_table)
    in_scope = silver.join(scope, "region").filter(F.col("in_scope"))
    return
in_scope.groupBy("wave_id").agg(F.sum("final_weight").alias("weighted_base"))
```

Line 4 hardcodes a production table, so every environment reads production scope rules. Add `scope_table` to the config and read it from `cfg`.

Common wrong answers.

- Line 3: Line 3 reads the silver table from config, which is correct. Look for a name typed into the code.
- Line 5: The join and filter are fine. The problem is which table the join reads.